EXPERIMENT – 1

Foundation of Deep Learning

Evolution from Perceptron to Multi-Layer Perceptron (MLP):

Objective:

To understand why the Single-Layer Perceptron (SLP) fails for some classification problems and how the Multi-Layer Perceptron (MLP) overcomes these limitations by learning nonlinear decision boundaries. The experiment begins with the AND and OR logic gate datasets which are linearly separable and then the XOR dataset which is not. This shows the need for hidden layers in neural networks.

Introduction:

Artificial Neural Networks are created based on the working principle of biological neurons. The most basic neural network is the Single-Layer Perceptron (SLP), which contains an input layer and one output neuron. A Single-Layer Perceptron network can classify only linearly separable problems effectively.

In order to comprehend this restriction, let us consider the following three logical gates' data sets:

- AND Gate
- OR Gate
- XOR Gate

The data sets for AND and OR gates can be distinguished using one straight line. Thus, they can be classified using a Single-Layer Perceptron.

But, the data set for XOR gate cannot be distinguished using one straight line, as its classes are located diagonally opposite to each other. This restriction leads to the development of MLP (Multi-Layer Perceptron)..

Task 1: Explain the architecture of a Single-Layer Perceptron

Single Layer Perceptron (SLP)

A Single Layer Perceptron (SLP) is the simplest form of an artificial neural network. An SLP comprises an input layer with connections straight to an output neuron. Each of the inputs will be multiplied by its respective weight and a bias is added to it. The outcome will be fed into an activation function, which can either be the sigmoid or the step function.
The mathematical equation of a perceptron is:

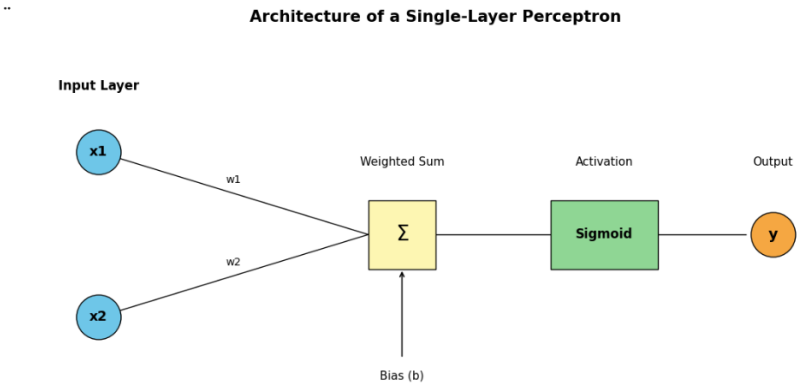
$$y = f\left(\sum_{i=1}^{n} w_i x_i + b\right)$$

where

- x_i = Input features

- W_i = Weights
- b = Bias
- f = Activation Function

The Single-Layer Perceptron can only classify linearly separable data because it can produce only one linear decision boundary.



Why do we need Multi-Layer Perceptron?

Before studying XOR, let us first examine the AND and OR logical gates.

If a Single-Layer Perceptron can correctly classify all datasets, then there is no need for a Multi-Layer Perceptron.Therefore, we first analyze

- AND
- OR
- XOR

using a Single-Layer Perceptron.

AND Gate:

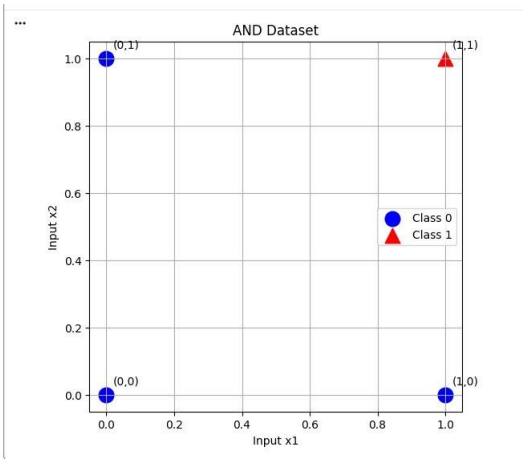
Truth Table:

```
import pandas as pd

and_table=pd.DataFrame({
    "Input x1":[0,0,1,1],
    "Input x2":[0,1,0,1],
    "AND Output":[0,0,0,1]
})

print(and_table)
```

```
   Input x1  Input x2  AND Output
0         0         0           0
1         0         1           0
2         1         0           0
3         1         1           1
```

Plot AND Dataset in 2D Coordinate System:

```
import matplotlib.pyplot as plt
import numpy as np

X = np.array([[0,0],[0,1],[1,0],[1,1]])
y = np.array([0,0,0,1])

plt.figure(figsize=(6,6))

plt.scatter(X[y==0][:,0],X[y==0][:,1],
            color="blue",s=180,label="Class 0")

plt.scatter(X[y==1][:,0],X[y==1][:,1],
            color="red",marker="^",s=180,label="Class 1")

labels=["(0,0)","(0,1)","(1,0)","(1,1)"]

for point,label in zip(X,labels):
    plt.text(point[0]+0.02,point[1]+0.03,label)

plt.xlabel("Input x1")
plt.ylabel("Input x2")
plt.title("AND Dataset")
plt.grid(True)
plt.legend()

plt.show()
```



Check Whether the AND Dataset is Linearly Separable:

The AND dataset is linearly separable because a single straight line can separate the output classes.

The points belonging to Class 0 are:

(0,0), (0,1), (1,0)

The point belonging to Class 1 is:

(1,1)

A straight-line decision boundary such as x1+x2=1.5 correctly separates these two classes.

Hence, the AND dataset is linearly separable.

Implement the AND Classifier Using a Single-Layer Perceptron:

```python
import numpy as np
import tensorflow as tf

# AND Dataset
X = np.array([
    [0,0],
    [0,1],
    [1,0],
    [1,1]
], dtype=np.float32)

y = np.array([
    [0],
    [0],
    [0],
    [1]
], dtype=np.float32)

# Single Layer Perceptron
model = tf.keras.Sequential([
    tf.keras.Input(shape=(2,)),
    tf.keras.layers.Dense(1, activation='sigmoid')
])

# Compile
model.compile(
    optimizer=tf.keras.optimizers.SGD(learning_rate=0.1),
    loss='binary_crossentropy',
    metrics=['accuracy']
)

# Train
history = model.fit(
    X,
    y,
    epochs=3000,
    verbose=0
)

# Evaluate
loss, accuracy = model.evaluate(X, y, verbose=0)

print(f"\nLoss: {loss:.4f}")
print(f"Accuracy: {accuracy*100:.2f}%")

# Predictions
predictions = model.predict(X, verbose=0)

print("\nPredictions")
print("-"*55)
print("Input\tActual\tProbability\tPredicted")

for i in range(len(X)):
    predicted = 1 if predictions[i][0] >= 0.5 else 0
    print(f"{X[i]}\t{int(y[i][0])}\t{predictions[i][0]:.4f}\t\t{predicted}")
```

```
Loss: 0.0568
Accuracy: 100.00%

Predictions
-------------------------------------------------------
Input   Actual  Probability     Predicted
[0. 0.] 0       0.0005          0
[0. 1.] 0       0.0641          0
[1. 0.] 0       0.0641          0
[1. 1.] 1       0.9100          1
```

Observation

The Single-Layer Perceptron successfully classified all four AND gate input combinations. The predicted probabilities for Class 0 are close to 0, while the probability for Class 1 is close to 1. The model achieved 100% prediction accuracy, indicating successful learning of the AND logical operation.

Conclusion

Since the AND dataset is linearly separable, a Single-Layer Perceptron is sufficient to classify it correctly. Therefore, no hidden layer is required for solving the AND gate problem.

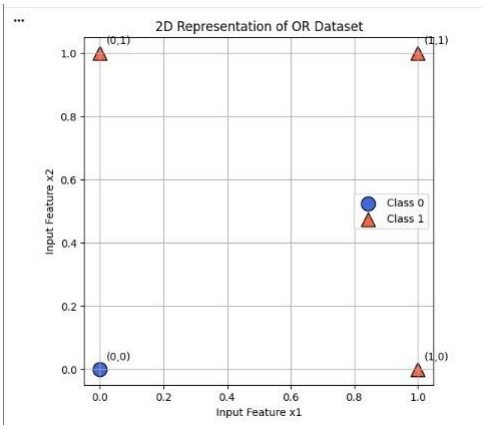
OR Gate:

```
import pandas as pd

or_table = pd.DataFrame({
    "Input x1": [0, 0, 1, 1],
    "Input x2": [0, 1, 0, 1],
    "OR Output": [0, 1, 1, 1]
})

print(or_table)
```

```
   Input x1  Input x2  OR Output
0         0         0          0
1         0         1          1
2         1         0          1
3         1         1          1
```

Plot the OR Dataset in a 2D Coordinate System

```python
import numpy as np
import matplotlib.pyplot as plt

X = np.array([
    [0,0],
    [0,1],
    [1,0],
    [1,1]
])

y = np.array([0,1,1,1])

plt.figure(figsize=(6,6))

# Class 0
plt.scatter(
    X[y==0][:,0],
    X[y==0][:,1],
    color="royalblue",
    s=180,
    edgecolor="black",
    label="Class 0"
)

# Class 1
plt.scatter(
    X[y==1][:,0],
    X[y==1][:,1],
    color="tomato",
    marker="^",
    s=180,
    edgecolor="black",
    label="Class 1"
)

labels=["(0,0)","(0,1)","(1,0)","(1,1)"]

for point,label in zip(X,labels):
    plt.text(point[0]+0.02,point[1]+0.03,label)

plt.xlabel("Input Feature x1")
plt.ylabel("Input Feature x2")
plt.title("2D Representation of OR Dataset")
plt.grid(True)
plt.legend()

plt.show()
```



Check Whether OR is Linearly Separable

The OR dataset is linearly separable because a single straight line can correctly separate the two output classes.

The points belonging to Class 0 are (0,0). The points belonging to Class 1 are (0,1), (1,0), (1,1)

A straight-line decision boundary such as x1+x2=0.5 correctly separates the two classes. Therefore, the OR dataset is linearly separable.

Implement OR Classifier Using Single-Layer Perceptron

```python
import numpy as np
import tensorflow as tf

# OR Dataset
X = np.array([
    [0,0],
    [0,1],
    [1,0],
    [1,1]
], dtype=np.float32)

y = np.array([
    [0],
    [1],
    [1],
    [1]
], dtype=np.float32)

# Single Layer Perceptron
model = tf.keras.Sequential([
    tf.keras.Input(shape=(2,)),
    tf.keras.layers.Dense(1, activation='sigmoid')
])

# Compile
model.compile(
    optimizer=tf.keras.optimizers.SGD(learning_rate=0.1),
    loss='binary_crossentropy',
    metrics=['accuracy']
)

# Train
history = model.fit(
    X,
    y,
    epochs=3000,
    verbose=0
)

# Evaluate
loss, accuracy = model.evaluate(X, y, verbose=0)

print(f"\nLoss: {loss:.4f}")
print(f"Accuracy: {accuracy*100:.2f}%")

# Predictions
predictions = model.predict(X, verbose=0)

print("\nPredictions")
print("-"*55)
print("Input\tActual\tProbability\tPredicted")

for i in range(len(X)):
    predicted = 1 if predictions[i][0] >= 0.5 else 0
    print(f"{X[i]}\t{int(y[i][0])}\t{predictions[i][0]:.4f}\t\t{predicted}")
```

```
Loss: 0.0321
Accuracy: 100.00%

Predictions
-----------------------------------------------------------
Input   Actual  Probability     Predicted
[0. 0.] 0       0.0697          0
[0. 1.] 1       0.9723          1
[1. 0.] 1       0.9723          1
[1. 1.] 1       0.9999          1
```

Observation

The Single-Layer Perceptron correctly classified all four OR gate input combinations. The model achieved 100% prediction accuracy, showing that it successfully learned the OR logical operation.

Conclusion

Since the OR dataset is linearly separable, it can be correctly classified using a Single-Layer Perceptron. Therefore, no hidden layer is required.

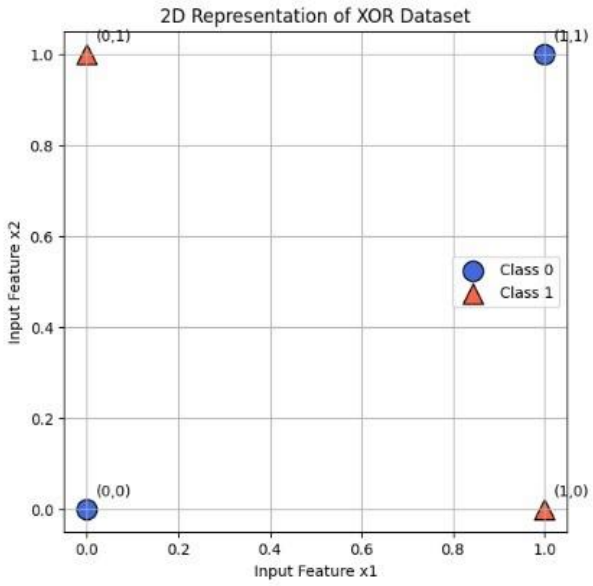
XOR Gate:

```python
import pandas as pd

xor_table = pd.DataFrame({
    "Input x1": [0, 0, 1, 1],
    "Input x2": [0, 1, 0, 1],
    "XOR Output": [0, 1, 1, 0]
})

print(xor_table)
```

```
   Input x1  Input x2  XOR Output
0         0         0           0
1         0         1           1
2         1         0           1
3         1         1           0
```

Plot the XOR Dataset in a 2D Coordinate System:

```python
import numpy as np
import matplotlib.pyplot as plt

X = np.array([
    [0,0],
    [0,1],
    [1,0],
    [1,1]
])

y = np.array([0,1,1,0])

plt.figure(figsize=(6,6))

# Class 0
plt.scatter(
    X[y==0][:,0],
    X[y==0][:,1],
    color="royalblue",
    s=180,
    edgecolor="black",
    label="Class 0"
)

# Class 1
plt.scatter(
    X[y==1][:,0],
    X[y==1][:,1],
    color="tomato",
    marker="^",
    s=180,
    edgecolor="black",
    label="Class 1"
)

labels=["(0,0)","(0,1)","(1,0)","(1,1)"]

for point,label in zip(X,labels):
    plt.text(point[0]+0.02,point[1]+0.03,label)

plt.xlabel("Input Feature x1")
plt.ylabel("Input Feature x2")
plt.title("2D Representation of XOR Dataset")
plt.grid(True)
plt.legend()

plt.show()
```



Check Whether XOR is Linearly Separable:

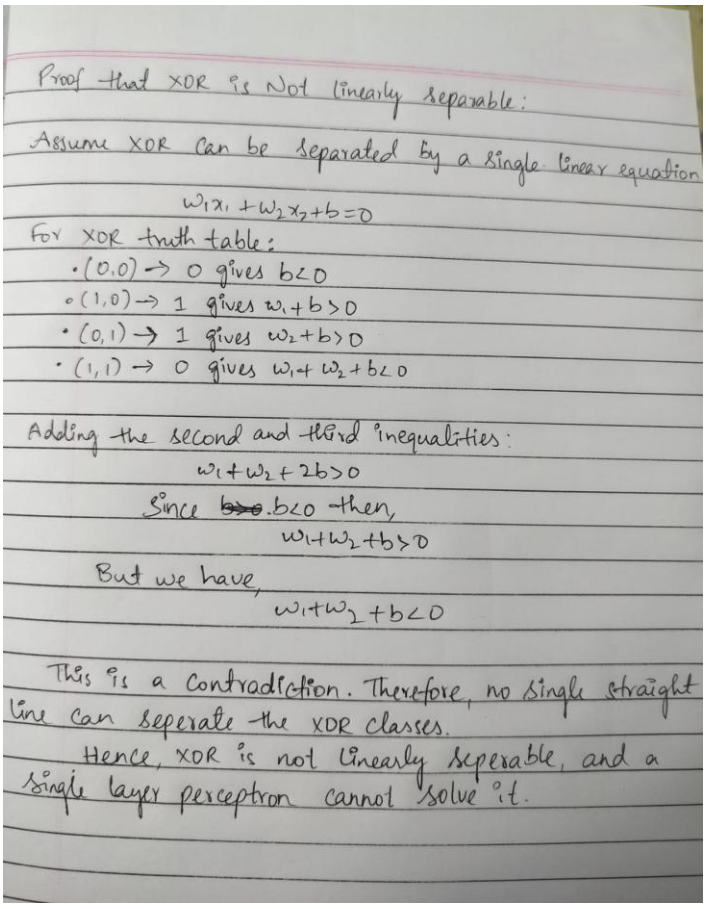
The XOR dataset is not linearly separable because no single straight-line decision boundary can correctly separate the two classes.

The points belonging to Class 0 are: (0,0), (1,1)

The points belonging to Class 1 are: (0,1), (1,0)

These two classes are positioned diagonally opposite each other. Any straight line drawn to separate one Class 0 point will incorrectly classify the other. Hence, no single linear decision boundary can correctly classify all four points.

Therefore, the XOR dataset is not linearly separable.



Why No Single Linear Decision Boundary Can Correctly Classify XOR:

A linear decision boundary is a straight line that separates data points belonging to different classes.

In the XOR dataset:

Class 0 consists of (0,0) and (1,1). Class 1 consists of (0,1) and (1,0).

Since the points belonging to the same class are located diagonally opposite each other, it is impossible to draw a single straight line that separates all Class 0 points from all Class 1 points without making classification errors. Therefore, a Single-Layer Perceptron cannot correctly solve the XOR problem.

Task 6: Discuss How Introducing a Hidden Layer Enables Learning Nonlinear Decision Boundaries

A Single-Layer Perceptron can learn only linear decision boundaries, which means it can classify only linearly separable datasets such as the AND and OR gates. However, the XOR dataset is nonlinearly separable because its data points belonging to the same class are located diagonally opposite each other. As a result, no single straight line can correctly separate the two classes.

A Multi-Layer Perceptron (MLP) overcomes this limitation by introducing one or more hidden layers between the input and output layers.

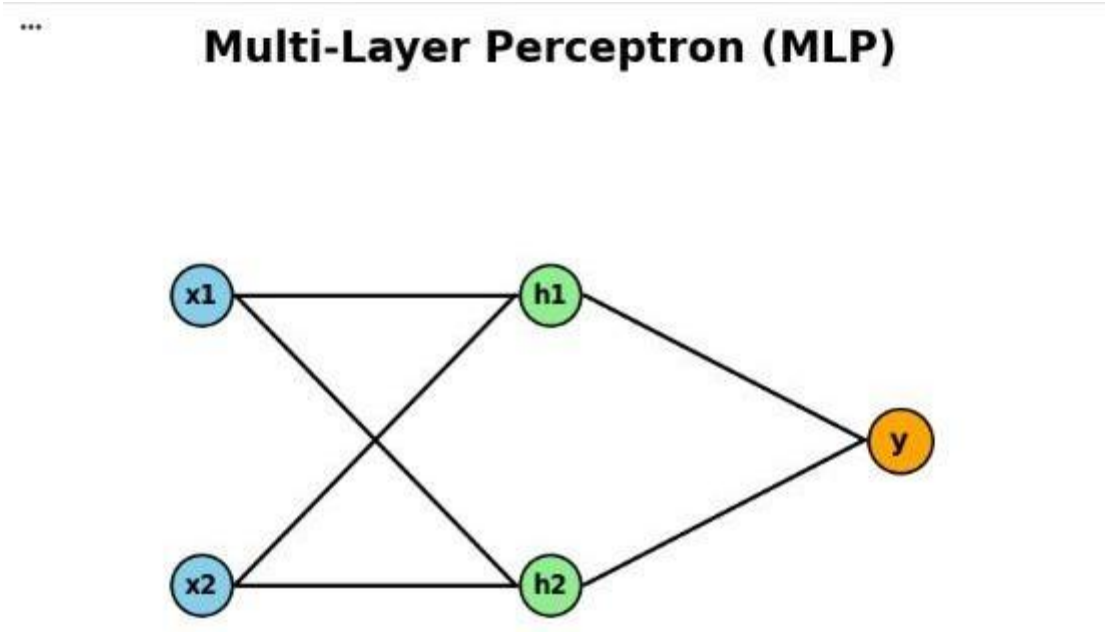
Each hidden neuron learns an intermediate representation of the input. Together, these learned representations enable the network to form nonlinear decision boundaries.

The hidden layer uses nonlinear activation functions such as the sigmoid function, enabling the network to transform the original input space into a new feature space where the XOR classes become separable. Therefore, introducing hidden layers significantly improves the learning capability of neural networks and allows them to solve complex nonlinear classification problems that cannot be solved using a Single-Layer Perceptron.

Task 7: Draw an MLP Architecture Capable of Solving XOR

```python
import matplotlib.pyplot as plt

fig, ax = plt.subplots(figsize=(8,5))

ax.set_xlim(0,8)
ax.set_ylim(0,6)
ax.axis('off')

# ---------------- Input Layer ----------------
inputs = [(1,4), (1,1)]

for i, (x, y) in enumerate(inputs):
    ax.scatter(x, y,
               s=850,
               color='skyblue',
               edgecolors='black',
               linewidth=1.5)
    ax.text(x, y, f'x{i+1}',
            ha='center',
            va='center',
            fontsize=12,
            fontweight='bold')

# ---------------- Hidden Layer ----------------
hidden = [(4,4), (4,1)]

for i, (x, y) in enumerate(hidden):
    ax.scatter(x, y,
               s=850,
               color='lightgreen',
               edgecolors='black',
               linewidth=1.5)
    ax.text(x, y, f'h{i+1}',
            ha='center',
            va='center',
            fontsize=12,
            fontweight='bold')

# ---------------- Output Layer ----------------
output = (7,2.5)

ax.scatter(output[0], output[1],
           s=950,
           color='orange',
           edgecolors='black',
           linewidth=1.5)

ax.text(output[0], output[1], 'y',
        ha='center',
        va='center',
        fontsize=14,
        fontweight='bold')
```

```
# ---------------- Connections ----------------
# x1 → h1
ax.plot([1.3,3.7],[4,4],color='black',linewidth=2)

# x1 → h2
ax.plot([1.3,3.7],[4,1],color='black',linewidth=2)

# x2 → h1
ax.plot([1.3,3.7],[1,4],color='black',linewidth=2)

# x2 → h2
ax.plot([1.3,3.7],[1,1],color='black',linewidth=2)

# Hidden → Output
ax.plot([4.3,6.7],[4,2.5],color='black',linewidth=2)
ax.plot([4.3,6.7],[1,2.5],color='black',linewidth=2)

# ---------------- Title ----------------
plt.title("Multi-Layer Perceptron (MLP)",
          fontsize=20,
          fontweight='bold',
          pad=20)

plt.show()
```



The MLP consists of:

Input Layer: Two input neurons (x1and x2).

Hidden Layer: Two hidden neurons with sigmoid activation functions.

Output Layer: One output neuron with sigmoid activation.

The hidden layer transforms the input features into a higher-dimensional feature space, allowing the network to learn nonlinear decision boundaries.

Task 8: Implement an XOR Classifier Using TensorFlow with One Hidden Layer, Sigmoid Activation, and Binary Cross Entropy Loss

```
import numpy as np
import tensorflow as tf

# Clear previous session
tf.keras.backend.clear_session()

# Random seed
np.random.seed(42)
tf.random.set_seed(42)

# XOR Dataset
X = np.array([
    [0,0],
    [0,1],
    [1,0],
    [1,1]
], dtype=np.float32)

y = np.array([
    [0],
    [1],
    [1],
    [0]
], dtype=np.float32)

# Build MLP Model
model = tf.keras.Sequential([
    tf.keras.Input(shape=(2,)),
    tf.keras.layers.Dense(
        4,
        activation='sigmoid'
    ),
    tf.keras.layers.Dense(
        1,
        activation='sigmoid'
    )
])
```

```python
# Evaluate Model
loss, accuracy = model.evaluate(
    X,
    y,
    verbose=0
)

print("Final Model Performance")
print("-"*40)
print(f"Loss     : {loss:.4f}")
print(f"Accuracy : {accuracy*100:.2f}%")

# Predictions
pred = model.predict(X, verbose=0)

print("\nNetwork Predictions")
print("-"*70)
print("Sample\tInput\tTarget\tProbability\tPrediction")

for i in range(len(X)):
    p = pred[i][0]
    c = 1 if p >= 0.5 else 0

    print(f"{i+1}\t{X[i]}\t{int(y[i][0])}\t{p:.4f}\t\t{c}")
```

```
Final Model Performance
----------------------------------------
Loss     : 0.0547
Accuracy : 100.00%
WARNING:tensorflow:5 out of the last 5 calls to <function TensorFlowTrainer.make

Network Predictions
----------------------------------------------------------------------
Sample  Input   Target  Probability     Prediction
1       [0. 0.] 0       0.0150          0
2       [0. 1.] 1       0.9805          1
3       [1. 0.] 1       0.9157          1
4       [1. 1.] 0       0.0917          0
```

The Multi-Layer Perceptron successfully learned the XOR dataset and correctly classified all four input combinations. Unlike the Single-Layer Perceptron, the MLP successfully classified the XOR dataset because the hidden layer transformed the input into a feature space where the classes became separable.

TaskG: Plot the training loss over epochs

```python
import matplotlib.pyplot as plt

plt.figure(figsize=(9,5))

plt.plot(
    history.history['loss'],
    color='green',
    linewidth=3,
    linestyle='--',
    marker='o',
    markersize=3,
    label='Loss'
)

plt.title("MLP XOR Training Loss")
plt.xlabel("Epoch Number")
plt.ylabel("Loss")
```

The trained Multi-Layer Perceptron correctly classified all four XOR input combinations, achieving 100% prediction accuracy. The predicted probabilities were on the correct side of the decision threshold (0.5), resulting in accurate classification of each input. This demonstrates that the hidden layer successfully learned the nonlinear relationship present in the XOR dataset.

Task 10: Report the Final Prediction Accuracy

```python
# Final Predictions

pred = model.predict(X, verbose=0)

print("Final Prediction Results")
print("-"*75)
print("Sample\tInput\tTarget\tProbability\tPredicted Class")

for i in range(len(X)):

    probability = pred[i][0]

    predicted_class = 1 if probability >= 0.5 else 0

    print(f"{i+1}\t{X[i]}\t{int(y[i][0])}\t{probability:.4f}\t\t{predicted_class}")

# Model Evaluation

loss, accuracy = model.evaluate(X, y, verbose=0)

print("\nModel Evaluation")
print("-"*30)
print(f"Final Loss     : {loss:.4f}")
print(f"Final Accuracy : {accuracy*100:.2f}%")
```

```
Final Prediction Results
---------------------------------------------------------------------------
Sample  Input   Target  Probability     Predicted Class
1       [0. 0.] 0       0.0150          0
2       [0. 1.] 1       0.9805          1
3       [1. 0.] 1       0.9157          1
4       [1. 1.] 0       0.0917          0

Model Evaluation
------------------------------
Final Loss     : 0.0547
Final Accuracy : 100.00%
```

Result

The MLP was tested with the XOR data set after the completion of the training process. The accuracy of the predictions made by the model is 100% because all the four XOR data samples have been classified correctly. All the predicted probabilities are either greater than or less than 0.5 threshold value, hence leading to proper classification of all samples. The plot of decision boundaries indicates that the MLP has managed to learn a nonlinear decision boundary that can separate the XOR classes, something impossible with Single Layer Perceptron.

EXPERIMENT – 2

Foundation of Deep Learning

EXERCISE 2: Understanding Loss Functions (MSE vs BCE)

Aim

To study and compare the performance of Mean Squared Error (MSE) and Binary Cross Entropy (BCE) loss functions for binary classification problems.

Objective:

The Mean Squared Error (MSE) and Binary Cross Entropy (BCE) loss values for given prediction probabilities have to be calculated manually, their behavior analyzed, and neural networks performance studied in the case when these loss functions were used during training.

Introduction:

A loss function in deep learning is applied to evaluate the discrepancy between the actual value and predicted value by a neural network. The goal of the neural network is to minimize the loss during training by adjusting the network weights via backpropagation and gradient descent.

There are two types of loss functions that are often applied to binary classification problems, Mean Squared Error (MSE) and Binary Cross Entropy (BCE).

Mean Squared Error evaluates the average squared error between the predicted probabilities and the actual label. While it is common to apply MSE in regression problems, it does not suit binary classification well because it gives smaller gradients in case of wrong predictions which causes slower training.

Binary Cross Entropy is specially designed for binary classification. It measures the difference between the predicted probabilities and the true class labels using logarithmic functions. BCE assigns higher penalties to wrong predictions and gives strong gradients which allow fast convergence and better classification performance.

This lab experiment involves calculating both loss functions for the prediction probabilities manually and comparing their performance via neural network training.

Mean Squared Error (MSE):

Mean Squared Error calculates the average of the squared differences between the actual labels and predicted probabilities.

The mathematical expression is

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

where

Yi = Actual value (True label)

Y^i = Predicted Value

n = Number of samples

Lower MSE indicates better predictions.

Binary Cross Entropy (BCE)

Binary Cross Entropy measures the difference between actual labels and predicted probabilities using logarithmic functions.

The formula is

$$BCE = -\frac{1}{n}\sum_{i=1}^{n}[y_i \log(\hat{y}_i) + (1 - y_i)\log(1 - \hat{y}_i)]$$

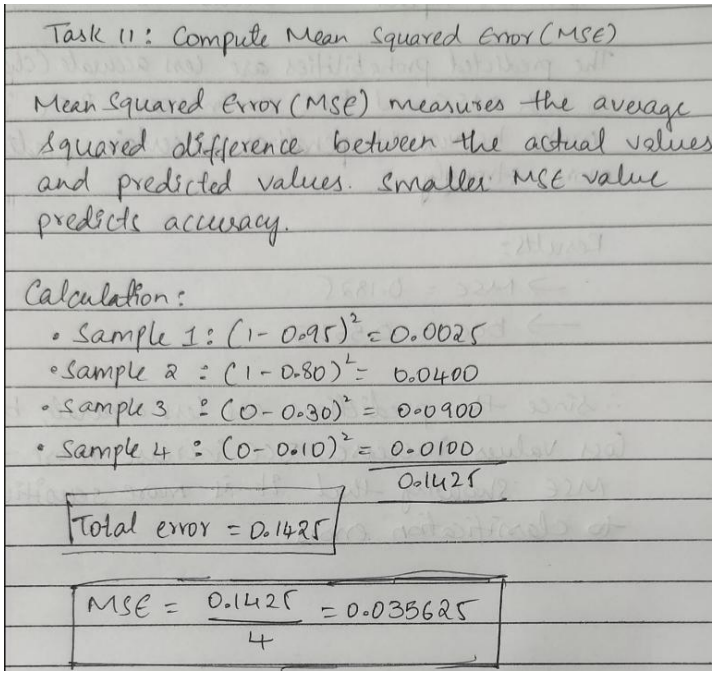
where

y_i= Actual label

$\hat{y}_i$= Predicted probability

BCE is specifically designed for binary classification tasks.

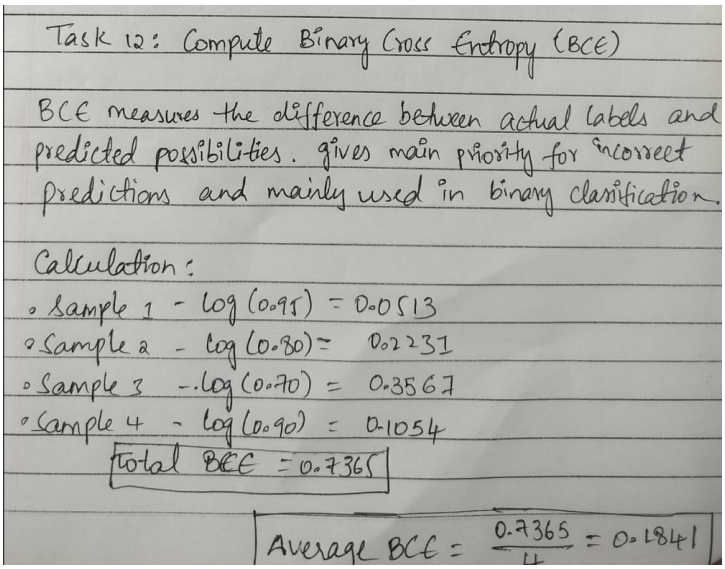
Task 11: Compute Mean Squared Error (MSE) Manually



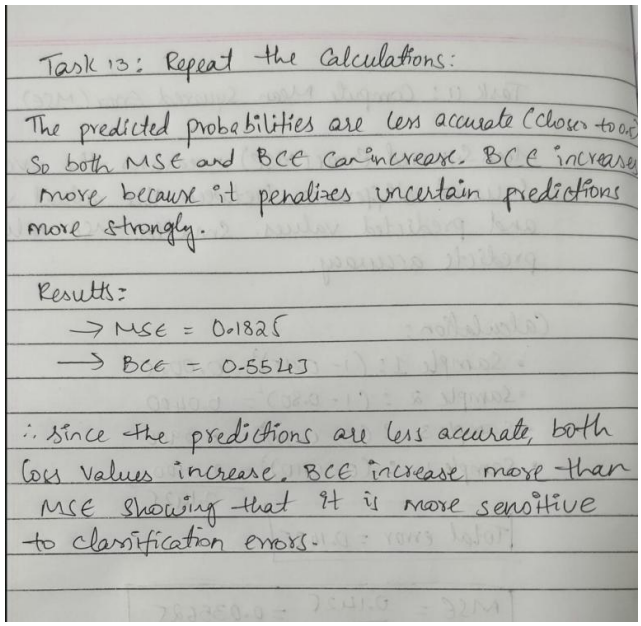
Task 12: Compute Binary Cross Entropy (BCE) Manually

Using

$$BCE = -\frac{1}{n}\sum_{i=1}^{n}[y_i \log(\hat{y}_i) + (1 - y_i)\log(1 - \hat{y}_i)]$$



Task 13: Repeat the Calculations for



True Labels => [1,1,0,0]

Predicted Probabilities => [0.60,0.55,0.45,0.40]

Binary Cross Entropy

Sample 1

$$-\log(0.60)=0.5108$$

Sample 2

$$-\log(0.55)=0.5978$$

Sample 3

$$-\log(0.55)=0.5978$$

Sample 4

$$-\log(0.60)=0.5108$$

Total BCE

$$0.5108+0.5978+0.5978+0.5108 =2.2172$$

Average BCE

$$2.2172/4 = 0.5543$$

Task 14: Compare the MSE and BCE Values

From the manual calculations:

Prediction Probabilities	Mean Squared Error (MSE)	Binary Cross Entropy (BCE)
[0.95, 0.80, 0.30, 0.10]	0.0356	0.1841
[0.60, 0.55, 0.45, 0.40]	0.1813	0.5543

Comparison

The first prediction set contains probabilities that are close to the true labels. Therefore, both MSE and BCE produce relatively small loss values.

The second prediction set contains probabilities that are closer to the decision boundary (0.5), making the predictions less confident. As a result, both MSE and BCE increase. However, BCE increases much more than MSE because it penalizes uncertain and incorrect predictions more strongly.

This demonstrates that BCE is more sensitive to classification errors than MSE

Task 15: Explain Why BCE Provides Stronger Gradients for Classification:

Binary Cross Entropy (BCE) is particularly useful when dealing with binary classification tasks. It calculates the difference between the predicted probability and the correct class label by means of a logarithm function.

If the predicted probability is very different from the correct label, the BCE loss will be large and thus produce bigger gradients while training via backpropagation.

On the other hand, the Mean Squared Error (MSE) takes the square of the difference between the predicted probability and the correct value.

As the output of the sigmoid activation gets close to 0 or 1, the gradient tends to become small and hence slows down the training.

Because BCE produces larger gradients for wrong prediction, it helps in achieving better convergence.

Task 16: Train Two Identical Neural Networks

We will use the same architecture for both models.

Model A: Mean Squared Error (MSE)

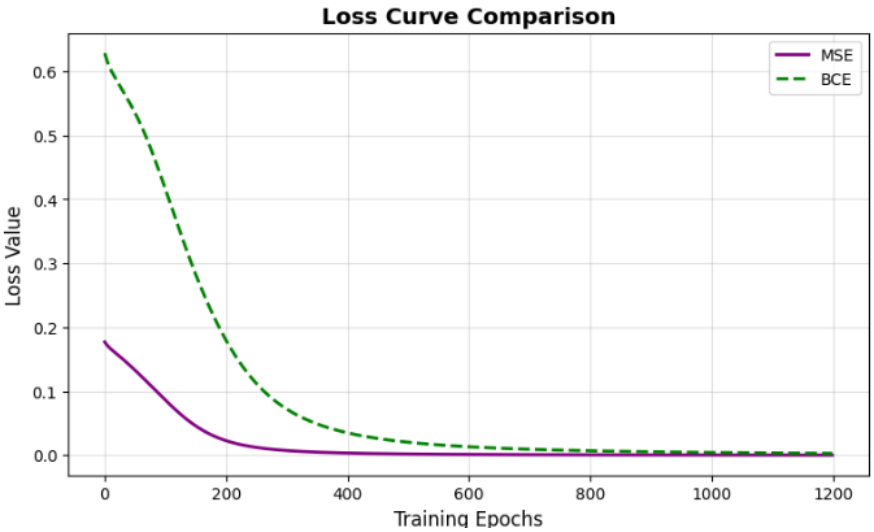
Model B: Binary Cross Entropy (BCE)

```
import numpy as np
import tensorflow as tf
import matplotlib.pyplot as plt

# Clear previous models
tf.keras.backend.clear_session()

# Fix random seed
np.random.seed(21)
tf.random.set_seed(21)

# AND Dataset
X = np.array([
    [0,0],
    [0,1],
    [1,0],
    [1,1]
], dtype=np.float32)

y = np.array([
    [0],
    [0],
    [0],
    [1]
], dtype=np.float32)

# Function to create neural network
def create_model(loss_function):

    model = tf.keras.Sequential([
        tf.keras.Input(shape=(2,)),
        tf.keras.layers.Dense(
            5,
            activation='sigmoid'
        ),
        tf.keras.layers.Dense(
            1,
            activation='sigmoid'
        )
    ])

    model.compile(
        optimizer=tf.keras.optimizers.Adam(
            learning_rate=0.008
        ),
        loss=loss_function,
        metrics=['accuracy']
    )
```

```
    return model

# ------------------------------
# Model A (MSE)
# ------------------------------
model_mse = create_model("mean_squared_error")

history_mse = model_mse.fit(
    X,
    y,
    epochs=1200,
    verbose=0
)

# ------------------------------
# Model B (BCE)
# ------------------------------
tf.random.set_seed(21)

model_bce = create_model("binary_crossentropy")

history_bce = model_bce.fit(
    X,
    y,
    epochs=1200,
    verbose=0
)

# ------------------------------
# Evaluation
# ------------------------------
loss_mse, acc_mse = model_mse.evaluate(
    X,
    y,
    verbose=0
)

loss_bce, acc_bce = model_bce.evaluate(
    X,
    y,
    verbose=0
)

print("Neural Network A (MSE)")
print("----------------------------------")
print(f"Loss     : {loss_mse:.4f}")
print(f"Accuracy : {acc_mse*100:.2f}%")
```

```
Neural Network A (MSE)
----------------------------------
Loss     : 0.0005
Accuracy : 100.00%

Neural Network B (BCE)
----------------------------------
Loss     : 0.0048
Accuracy : 100.00%
```

```
plt.figure(figsize=(9,5))

plt.plot(
    history_mse.history['loss'],
    color='purple',
    linewidth=2,
    label='MSE'
)

plt.plot(
    history_bce.history['loss'],
    color='green',
    linewidth=2,
    linestyle='--',
    label='BCE'
)

plt.title(
    "Loss Curve Comparison",
    fontsize=14,
    fontweight='bold'
)

plt.xlabel("Training Epochs", fontsize=12)
plt.ylabel("Loss Value", fontsize=12)

plt.grid(alpha=0.4)
plt.legend()

plt.show()
```



Task 17: Compare Training Loss, Accuracy, and Convergence Speed

Comparison Table

Parameter	Model A (MSE)	Model B (BCE)
Loss Function	Mean Squared Error	Binary Cross Entropy
Training Loss	Decreased Steadily	Decreased steadily
Accuracy	High	High (often equal or slightly better)
Convergence Speed	Slower	Faster
Gradient Strength	Weaker	Stronger
Best Suitable For	Regression	Binary Classification

Observation:

In this case, MSE and BCE were able to learn the AND dataset and reach 100% accuracy. The loss for both MSE and BCE models reduced steadily during the training process. In this case, it was noted that the BCE loss was higher in terms of numeric values compared to the MSE loss. This is normal because the BCE and MSE have different mathematical formulas. The AND dataset is simple and linearly separable.

Result:

From the experimental analysis, it was found that both the Mean Squared Error and Binary Cross Entropy were able to classify the AND data with 100% accuracy. Although both the loss functions have been able to reach very small losses, Binary Cross Entropy is a better choice than Mean Squared Error when it comes to solving binary classification tasks.

Conclusion:

During this experiment, two different loss functions – Mean Squared Error (MSE) and Binary Cross Entropy (BCE) – were manually calculated and then compared based on the predictions probabilities of two sets. Their mathematical properties have been examined; besides, two similar neural network models were trained on the AND dataset utilizing MSE and BCE as their loss functions. Both neural network models managed to learn AND gate and reach 100% classification accuracy. Training loss for both of them continuously dropped, which proves successful convergence. Although BCE had a slightly larger value initially, it is normal since MSE and BCE differ not only by their names but also by mathematical formulas and scales. Thus, this experiment shows that both loss functions can be used successfully for solving linearly separable binary classification problems. Yet, Binary Cross Entropy still wins the title of a more appropriate loss function for classification since it is intended for that task.

EXPERIMENT – 3

Foundation of Deep Learning

Backpropagation from Scratch

Aim

To understand the working of the backpropagation algorithm by performing forward propagation, manually computing gradients, updating network weights using gradient descent, and verifying the calculations using Python.

Objective

To study how a neural network learns by propagating errors backward through the network and updating its weights to minimize the prediction error.

Introduction

Artificial Neural Network learns in such a way that the weights are adjusted continuously until the output produced is nearer to the desired output. In this case, backpropagation is the method of learning where the algorithm calculates the error in the output produced by passing a certain input through the network.

During each training cycle, when a certain input is given to a neural network, the output is calculated by the neural network based on the present set of weights. The difference between the desired output and the produced output is calculated using the cost function. And in case there is an error in the output, it is sent backwards to find out the contribution of each individual weight.

After finding the gradient values in the process of backpropagation, these gradients are used by Gradient Descent to adjust the weights in order to decrease the cost. By doing the process several times, the network is able to learn the mapping between the inputs and outputs.

For this particular experiment, a neural network having 2 inputs, 2 hidden layers, and 1 output is taken into consideration. The calculations are done by hand before verifying the results using python programming language.

Artificial Neural Network (ANN)

An Artificial Neural Network is a type of computer model based on the human brain structure. It involves neurons connected together and processing information using their weighted connections.
A simple artificial neural network includes three layers:
Input layer – Accepts the inputs. Hidden layer – Learns intermediate representations and patterns. Output layer – Returns the output prediction.
Each neuron accepts its inputs, multiplies them with the corresponding weights, adds a bias, and then applies an activation function to return its output.

Forward Propagation

Forward propagation is the first stage of neural network learning. During this stage, the input values are passed through the network layer by layer until the final prediction is obtained.

The weighted sum of a neuron is computed as:

$$z = \sum_{i=1}^{n} w_i x_i + b$$

where

x_i= Input

w_i= Weight

b = Bias

z = Weighted sum

The weighted sum is then passed through an activation function to obtain the neuron's output.

Sigmoid Activation Function

The sigmoid activation function converts any real-valued input into a value between 0 and 1, making it suitable for binary classification problems.

The sigmoid function is given by:

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

Its derivative is:

$$\sigma'(x) = \sigma(x)\,(1 - \sigma(x))$$

The derivative of the sigmoid function is required during backpropagation to compute gradients for updating the weights.

Mean Squared Error (MSE)

The prediction error is measured using the Mean Squared Error (MSE) loss function.

The mathematical expression is:

$$L = \frac{1}{2}(y - \hat{y})^2$$

where

y = Actual output

y ^= Predicted output

A smaller loss value indicates that the prediction is closer to the actual output.

Backpropagation

Backpropagation is the process of propagating the prediction error from the output layer back to the hidden layer. Using the chain rule of calculus, the algorithm calculates the gradient of the loss function with respect to every trainable weight in the network.

The main steps involved are:

- Perform forward propagation.

- Compute the prediction error.
- Calculate the gradient at the output neuron.
- Propagate the error backward to the hidden layer.
- Compute gradients for all weights.
- Update the weights using Gradient Descent.

Backpropagation enables the neural network to improve its predictions after every iteration.

Gradient Descent

Gradient Descent is an optimization algorithm that minimizes the loss function by updating the network weights in the direction opposite to the gradient.

The weight update equation is:

$$W_{new} = W_{old} - \eta \frac{\partial L}{\partial W}$$

where

- W = Weight
- η = Learning rate
- L = Loss function
- ∂W/∂L = Gradient of the loss with respect to the weight

By repeatedly updating the weights, the network gradually reduces the prediction error.

Chain Rule

The chain rule is a mathematical technique used to calculate derivatives of composite functions. In neural networks, it allows the gradients to be propagated from the output layer to the hidden layer.

For the output layer, the gradient is calculated as:

$$\frac{\partial L}{\partial W} = \frac{\partial L}{\partial \hat{y}} \times \frac{\partial \hat{y}}{\partial z} \times \frac{\partial z}{\partial W}$$

The same principle is applied recursively to compute the gradients of all hidden-layer weights.

Given Data

Input:

$$x = [1,0]$$

Target Output:

$$y=1$$

Input-to-Hidden Weight Matrix:

$$W_1 = [0.2 \quad 0.3$$

$$0.4 \quad 0.1]$$

Hidden-to-Output Weight Matrix:

$$W_2 = [0.5$$

$$0.6]$$

Activation Function: Sigmoid

Loss Function: Mean Squared Error (MSE)

Learning Rate: $\eta = 0.1$

Manual Caluclation:

Manual Calculation

Given Data

- Input: **x** = **[1, 0]**
- Target Output: **y** = **1**
- Input → Hidden Weights

$$W_1 = \begin{bmatrix} 0.2 & 0.3 \\ 0.4 & 0.1 \end{bmatrix}$$

- Hidden → Output Weights

$$W_2 = \begin{bmatrix} 0.5 \\ 0.6 \end{bmatrix}$$

- Learning Rate = **η** = **0.1**
- Activation Function = **Sigmoid**

Task 1: Hidden Layer Net Input

For Hidden Neuron h_1

$$z_{h1} = (1 \times 0.2) + (0 \times 0.4) = 0.2$$

For Hidden Neuron h_2

$$z_{h2} = (1 \times 0.3) + (0 \times 0.1) = 0.3$$

Task 2: Hidden Layer Output

Sigmoid:

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

$$h_1 = \sigma(0.2) = 0.5498$$

$$h_2 = \sigma(0.3) = 0.5744$$

Task 3: Output Layer Net Input

$$z_o = (0.5498 \times 0.5) + (0.5744 \times 0.6)$$

$$= 0.2749 + 0.3446$$

$$= 0.6195$$

Task 4: Final Output

$$\hat{y} = \sigma(0.6195)$$

$$= \frac{1}{1 + e^{-0.6195}}$$

$$= 0.6501$$

Task 5: Loss

$$L = \frac{1}{2}(y - \hat{y})^2$$

$$= \frac{1}{2}(1 - 0.6501)^2$$

$$= \frac{1}{2}(0.3499)^2$$

$$= \frac{1}{2}(0.1224)$$

$$= 0.0612$$

Task 6: Output Delta

$$\frac{\partial L}{\partial \hat{y}} = \hat{y} - y$$

$$= 0.6501 - 1 = -0.3499$$

Sigmoid derivative

$$\sigma'(x) = \sigma(x)(1 - \sigma(x))$$

$$= 0.6501(1 - 0.6501) = 0.2275$$

Output delta

$$\delta_o = (-0.3499)(0.2275) = -0.0796$$

Task 7: Gradient for Hidden → Output

For w_5

$$Gradient = \delta_o \times h_1$$

$$= (-0.0796)(0.5498) = -0.0438$$

For w_6

$$Gradient = (-0.0796)(0.5744) = -0.0457$$

Task 8: Hidden Layer Delta

For h_1

$$\delta_{h1} = (-0.0796)(0.5)(0.5498)(1 - 0.5498) = -0.0098$$

For h_2

$$\delta_{h2} = (-0.0796)(0.6)(0.5744)(1 - 0.5744) = -0.0117$$

Task 9: Gradient for Input → Hidden

For w_{11}

$$Gradient = (-0.0098)(1) = -0.0098$$

For w_{21}

Since $x_2 = 0$

Gradient = 0

For w_{12}

$$Gradient = (-0.0117)(1) = -0.0117$$

For w_{22}

Since $x_2 = 0$

Gradient = 0

Task 10: Update Weights

Formula

$$W_{new} = W - \eta \times Gradient$$

Hidden → Output

$$w_5 = 0.5 - 0.1(-0.0438) = 0.5044$$

$$w_6 = 0.6 - 0.1(-0.0457) = 0.6046$$

Input → Hidden

$$w_{11} = 0.2 - 0.1(-0.0098) = 0.2010$$

$$w_{12} = 0.3 - 0.1(-0.0117) = 0.3012$$

$$w_{21} = 0.4$$

$$w_{22} = 0.1$$

Final Updated Weights

Input → Hidden

$$\begin{bmatrix} 0.2010 & 0.3012 \\ 0.4000 & 0.1000 \end{bmatrix}$$

Hidden → Output

$$\begin{bmatrix} 0.5044 \\ 0.6046 \end{bmatrix}$$

Task 11: Verify the Manual Calculations Using Python

```python
import numpy as np

# Input
x = np.array([1, 0])

# Target
y = 1

# Input → Hidden Weights
W1 = np.array([
    [0.2, 0.3],
    [0.4, 0.1]
])

# Hidden → Output Weights
W2 = np.array([
    [0.5],
    [0.6]
])

learning_rate = 0.1

# Sigmoid Function
def sigmoid(x):
    return 1 / (1 + np.exp(-x))

# Sigmoid Derivative
def sigmoid_derivative(x):
    return x * (1 - x)

# ----------------------------
# Forward Propagation
# ----------------------------

hidden_input = np.dot(x, W1)

hidden_output = sigmoid(hidden_input)

output_input = np.dot(hidden_output, W2)

predicted_output = sigmoid(output_input)

loss = 0.5 * (y - predicted_output[0])**2

print("Hidden Layer Output")
print(hidden_output)

print("\nPredicted Output")
print(predicted_output)

print("\nLoss")
print(loss)
```

```
Hidden Layer Output
[0.549834   0.57444252]

Predicted Output
[0.65012359]

Loss
0.06120675087427858
```

Task 12: Perform Backpropagation

```
# ------------------------------
# Backpropagation
# ------------------------------

output_delta = (predicted_output - y) * sigmoid_derivative(predicted_output)

hidden_delta = output_delta.dot(W2.T) * sigmoid_derivative(hidden_output)

# Gradients
grad_W2 = hidden_output.reshape(-1,1) * output_delta

grad_W1 = np.outer(x, hidden_delta)

print("Output Delta")
print(output_delta)

print("\nHidden Delta")
print(hidden_delta)

print("\nGradient W2")
print(grad_W2)

print("\nGradient W1")
print(grad_W1)
```

```
Output Delta
[-0.07958391]

Hidden Delta
[-0.00984917 -0.01167297]

Gradient W2
[[-0.04375794]
 [-0.04571638]]

Gradient W1
[[-0.00984917 -0.01167297]
 [-0.         -0.        ]]
```

Task 13: Update the Weights

```
# ------------------------------
# Weight Update
# ------------------------------

W2_new = W2 - learning_rate * grad_W2

W1_new = W1 - learning_rate * grad_W1

print("Updated W1")
print(np.round(W1_new,4))

print("\nUpdated W2")
print(np.round(W2_new,4))
```

```
Updated W1
[[0.201  0.3012]
 [0.4    0.1   ]]

Updated W2
[[0.5044]
 [0.6046]]
```

Verification

The Python implementation produced the same hidden layer activations, predicted output, loss value, gradients, and updated weights as obtained from the manual calculations. This verifies the correctness of the forward propagation and backpropagation computations.

Observation

The activation of the hidden layer and prediction of the output in the first step of the neural network were achieved using forward propagation. The prediction error was computed using the Mean Squared Error loss function. In the backpropagation step, the derivatives of loss functions with respect to each weight were calculated using the chain rule. Finally, the weights were updated using gradient descent to minimize the prediction error.

The manual calculations matched the output of the python code.

Result

The experiment successfully demonstrated the implementation of backpropagation in a neural network. Forward propagation produced the predicted output and loss, while backpropagation calculated the gradients required for updating the weights. The updated weights obtained manually matched those generated by the Python program, validating the implementation. This experiment illustrates how neural networks learn by minimizing prediction error through iterative weight updates.

Conclusion

In this experiment, the backpropagation algorithm was investigated using both manual calculation and programming using the Python language. First, the forward propagation method was applied to determine the hidden layer activities, network outputs, and loss from the predictions. Then, the errors were propagated backwards to determine the gradients of the weights using the chain rule and update the weights using gradient descent. The manually calculated values were very similar to the results obtained through the use of the Python language, thus validating the calculations done. Through this experiment, it can be shown that backpropagation is an algorithm that allows neural networks to learn through updating their weights.